

■ Depuis une Image

Docker Hub / registre public

```
oc new-app  
--image=docker.io/nginx:latest
```

■ Depuis Source (S2I)

Build automatique depuis Git

```
oc new-app  
python:3.9~https://github.com/...
```

■ Depuis Dockerfile

Build depuis un repo Git

```
oc new-app --strategy=docker
```

■ Depuis Catalogue

Templates pré-définis OpenShift

```
oc new-app  
--template=mysql-ephemeral -p ...
```

■ Depuis YAML/JSON

Déploiement déclaratif complet

```
oc apply -f manifest.yaml
```

■ Depuis Operator Hub

Opérateurs certifiés pour apps complexes

```
oc apply -f subscription.yaml
```

Depuis une Image

Docker Hub / registre public

```
$ oc new-app --image=docker.io/nginx:latest
```

OC CLI

- 1 Se connecter à la sandbox**
`oc login --token=
--server=https://api.sandbox...`
- 2 Créer un projet**
`oc new-project lab-image`
- 3 Déployer nginx depuis Docker Hub**
`oc new-app --image=docker.io/nginx:latest
--name=mon-nginx`
- 4 Exposer le service**
`oc expose svc/mon-nginx`
- 5 Obtenir l'URL de la route**
`oc get route mon-nginx`

Console Web

- 1 +Add**
Aller sur +Add → Container images
- 2 Image**
Entrer l'image : `docker.io/nginx:latest`
- 3 Config**
Nommer l'app, cocher Create a route
- 4 Create**
Cliquer Create, attendre le pod Running
- 5 Route**
Dans Topology, cliquer la flèche pour ouvrir l'URL

Conseil Utilisez `oc get pods` et `oc logs` pour vérifier l'état du conteneur.

Depuis Source (S2I)

Build automatique depuis Git

```
$ oc new-app python:3.9~https://github.com/...
```

OC CLI

- 1 Créer un projet**
`oc new-project lab-s2i`
- 2 Déployer depuis GitHub**
`oc new-app python:3.9~https://github.com/openshift/django-ex.git --name=mon-s2i`
- 3 Suivre le build**
`oc logs -f bc/mon-s2i`
- 4 Attendre le déploiement**
`oc rollout status deployment/mon-s2i`
- 5 Exposer le service**
`oc expose svc/mon-s2i`
- 6 Obtenir l'URL**
`oc get route mon-s2i`

Console Web

- 1 +Add**
Aller sur +Add → Import from Git
- 2 URL Git**
Entrer `https://github.com/openshift/django-ex`
- 3 Builder**
OpenShift détecte Python – vérifier le builder S2I
- 4 Config**
Nommer l'app et cocher Create a route
- 5 Create**
Cliquer Create, suivre le build dans Builds → BuildConfigs
- 6 Accéder**
Une fois le pod Running, cliquer la flèche dans Topology pour ouvrir l'URL

Conseil S2I injecte votre code dans une image de base — aucun Dockerfile requis !

Depuis Dockerfile

Build depuis un repo Git

```
$ oc new-app --strategy=docker
```

OC CLI

- 1 Créer un projet**
`oc new-project lab-dockerfile`
- 2 Déployer avec stratégie Docker**
`oc new-app --strategy=docker
https://github.com/sclorg/httpd-ex.git
--name=mon-docker-app`
- 3 Suivre le build**
`oc logs -f bc/mon-docker-app`
- 4 Attendre le déploiement**
`oc rollout status deployment/mon-docker-app`
- 5 Exposer le service**
`oc expose svc/mon-docker-app`
- 6 Vérifier la route**
`oc get route mon-docker-app`

Console Web

- 1 +Add**
Aller sur +Add → Import from Git
- 2 URL Git**
Entrer l'URL du repo contenant un Dockerfile
- 3 Strategy**
OpenShift détecte le Dockerfile – vérifier la stratégie
- 4 Config**
Nommer l'app et cocher Create a route
- 5 Create**
Cliquer Create – le build lit le Dockerfile
- 6 Suivi**
Suivre dans Topology, puis cliquer la flèche pour ouvrir l'URL

Conseil Le Dockerfile doit être à la racine du repo. Modifier Context dir dans les options avancées si besoin.

Depuis Catalogue

Templates pré-définis OpenShift

```
$ oc new-app --template=mysql-ephemeral -p ...
```

OC CLI

- 1 Créer un projet**
`oc new-project lab-catalog`
- 2 Lister les templates**
`oc get templates -n openshift`
- 3 Déployer MySQL**
`oc new-app --template=mysql-ephemeral -p
MYSQL_USER=user -p MYSQL_PASSWORD=pass -p
MYSQL_DATABASE=mydb`
- 4 Vérifier le pod**
`oc get pods`
- 5 Se connecter en interne**
`oc rsh mysql -u user -ppass mydb`

Console Web

- 1 +Add**
Aller sur +Add → Developer Catalog
- 2 Search**
Chercher MySQL dans la barre de recherche
- 3 Template**
Cliquer MySQL (Ephemeral)
- 4 Params**
Remplir user, password, database name
- 5 Create**
Cliquer Instantiate Template

Conseil Les templates créent DC, Service, Route et Secret en une seule opération.

Depuis YAML/JSON

Déploiement déclaratif complet

```
$ oc apply -f manifest.yaml
```

OC CLI

- 1 Créer un projet**
`oc new-project lab-yaml`
- 2 Créer le manifeste**
Créer `deployment.yaml` avec Deployment + Service + Route
- 3 Appliquer le manifeste**
`oc apply -f deployment.yaml`
- 4 Vérifier les ressources**
`oc get all`
- 5 Obtenir l'URL**
`oc get route`
- 6 Mettre à jour**
Modifier le YAML puis relancer `oc apply -f deployment.yaml`

Console Web

- 1 Import**
Cliquer l'icône + (Import YAML) en haut à droite
- 2 Éditeur**
Coller le manifeste YAML dans l'éditeur intégré
- 3 Validation**
La console valide la syntaxe en temps réel
- 4 Create**
Cliquer Create pour appliquer les ressources
- 5 Suivi**
Ressources visibles dans Topology et Workloads
- 6 Accéder**
Cliquer la flèche dans Topology pour ouvrir l'URL

Conseil Utilisez `oc explain deployment.spec` pour explorer la documentation des champs YAML.

Depuis Operator Hub

Opérateurs certifiés pour apps complexes

```
$ oc apply -f subscription.yaml
```

OC CLI

- 1 Lister les opérateurs**
`oc get packagemanifests -n openshift-marketplace | head -20`
- 2 Créer un projet**
`oc new-project lab-operator`
- 3 Créer une subscription**
`oc apply -f subscription.yaml`
- 4 Vérifier l'installation**
`oc get csv # ClusterServiceVersion`
- 5 Créer une instance**
Via le CRD de l'opérateur selon sa documentation

Console Web

- 1 OperatorHub**
Aller dans Operators → OperatorHub
- 2 Search**
Chercher Redis, Strimzi Kafka, Prometheus...
- 3 Install**
Cliquer Install – choisir namespace et canal
- 4 Status**
Attendre Succeeded dans Installed Operators
- 5 Instance**
Cliquer l'opérateur → Create instance

Conseil Sur la *sandbox Developer*, certains opérateurs nécessitent des droits *cluster-admin*.

Référence rapide — Commandes essentielles

Commande	Description
<code>oc login --token=... --server=...</code>	Connexion à la sandbox
<code>oc new-project <nom></code>	Créer un nouveau projet/namespace
<code>oc new-app --image=<image></code>	Déployer depuis une image Docker
<code>oc new-app <builder>~<git-url></code>	Déployer avec S2I depuis Git
<code>oc new-app --strategy=docker <git-url></code>	Build via Dockerfile depuis Git
<code>oc new-app --template=<nom> -p ...</code>	Déployer un template du catalogue
<code>oc apply -f manifest.yaml</code>	Déployer depuis un fichier YAML/JSON
<code>oc expose svc/<nom></code>	Créer une route publique
<code>oc get route</code>	Lister les routes et leurs URLs
<code>oc get pods</code>	Lister les pods et leur statut
<code>oc logs -f <pod></code>	Suivre les logs d'un pod en temps réel
<code>oc rollout status deployment/<nom></code>	Suivre l'état d'un déploiement
<code>oc rsh <pod></code>	Ouvrir un shell interactif dans un pod
<code>oc get all</code>	Lister toutes les ressources du projet
<code>oc get csv</code>	Lister les ClusterServiceVersions (Operators)
<code>oc delete project <nom></code>	Supprimer un projet complet